

# CSEN 296B-2 | Week 5.2 Lab Submission

## AstraNotes Requirements-to-UML Traceability Matrix

Student Name: Alex Shirazi

Date: April 30, 2026

Project: AstraNotes

### 1. Requirements-to-UML Traceability Matrix

The matrix covers the 8 most important AstraNotes requirements selected from the refined baseline (Week 3.1). Requirements are drawn from FR, NFR, and SPR series. All requirement wording is taken directly from the Week 3.1 refined baseline without paraphrase. UML evidence references the Week 4.2 design package.

| ID             | Requirement (summary)                                                                                                                                                                                                                    | Class / Object Evidence                                                                                                                                                                                   | Use Case / Activity Evidence                                                                                                                                                        | Deployment Evidence                                                                                                                | Status           | Gap Note                                                                                                                                                                                  |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-01          | The system shall allow a user to create a new text note with a non-empty title and body. Title validated by ValidationLayer; UUID, created_at, modified_at assigned automatically.                                                       | Note (id, title, body, created_at, modified_at); ValidationLayer (+validate_title, +validate_note); NoteService (+create())                                                                               | UC: Create Note; Activity: validation gate (empty title → ValidationError), assign UUID/timestamps node                                                                             | No deployment evidence needed purely application-layer logic.                                                                      | Fully Traced     | None. All structural, behavioral, and validation evidence is present and consistent.                                                                                                      |
| FR-04 / SPR-01 | The system shall allow a user to mark a note as private. Private notes shall be encrypted at rest using Fernet via PrivacyService. Encryption key shall not be stored in source code or data directory.                                  | SecureNote (inherits Note; +encrypt(), +decrypt(), -encrypted_body); PrivacyService (-_key, +encrypt(), +decrypt()); Object: secure1:SecureNote with encrypted_body =                                     | UC: Create Secure Note (includes Encrypt Body); UC: Unlock Secure Note (includes Decrypt Body); Activity: is_private branch → Create SecureNote → SecureNote.encrypt()              | Deployment: FERNET_KEY artifact labeled 'env var / OS keychain never in source code'. SecureNote+PrivacyService grouped component. | Fully Traced     | None. Strongest requirement in the package visible in all six diagram types.                                                                                                              |
| FR-05          | The system shall persist all notes to local file storage (one JSON file per note). On startup, all valid note files shall be loaded. Corrupt or missing files shall be skipped with an error logged.                                     | NoteRepository (interface: +save, +get, +list_all, +update, +delete); JsonFileRepository (-data_dir: Path, implements NoteRepository); Object: repo:JsonFileRepository with data_dir = ~/.astranotes/data | UC: Create Note (includes Persist to Disk); UC: View Sorted List (includes Load from Disk); Activity: NoteRepository.save(note) → Write JSON to data_dir → Write succeeds? decision | Deployment: {note_id}.json artifact at ~/.astranotes/data/ shown inside device node.                                               | Fully Traced     | Corrupt-file-on-startup behavior is noted in requirements but the activity diagram only shows the happy path and write failure. Load failure path is not shown in the activity diagram.   |
| FR-07          | The system shall allow keyword search across note titles and bodies. Search shall be case-insensitive. Private notes that decrypt successfully shall be included. Notes that cannot be decrypted shall be excluded with an error logged. | NoteService (+search(kw: str): List[Note]); PrivacyService (+decrypt()) search must route through PrivacyService for private notes.                                                                       | UC: Search Notes. No dedicated activity diagram for search flow.                                                                                                                    | No deployment evidence application-layer operation only.                                                                           | Partially Traced | Search is present as a use case and a NoteService method, but no activity diagram exists for the search flow. The private-note decrypt-on-search path has no behavioral diagram evidence. |

| ID     | Requirement (summary)                                                                                                                                                                                     | Class / Object Evidence                                                                                                                                                       | Use Case / Activity Evidence                                                                                                                            | Deployment Evidence                                                                                                                         | Status           | Gap Note                                                                                                                                                                                                                                           |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-08  | The system shall display notes sorted by modified_at descending by default. Notes with equal modified_at shall be ordered by created_at descending as a stable tiebreaker.                                | Note (-modified_at: datetime, -created_at: datetime); NoteService (+list_sorted(): List[Note]). Object: display:UI with sort = modified_at DESC.                              | UC: View Sorted List (includes Load from Disk).                                                                                                         | No deployment evidence needed.                                                                                                              | Partially Traced | Sorting is structurally supported (modified_at field, list_sorted method) and present as a use case. The stable tiebreaker logic (created_at) is not visible in any diagram it is implied by the field existing but never explicitly shown.        |
| NFR-01 | The system shall complete note listing and keyword search within 2 seconds for up to 500 notes on a machine with 8 GB RAM, SSD storage, and a modern dual-core CPU.                                       | No class or object in the design explicitly addresses performance. NoteService.list_sorted() and search() are the relevant methods but carry no performance annotation.       | No use case or activity diagram addresses the 2-second constraint.                                                                                      | No deployment evidence for the hardware baseline.                                                                                           | Weakly Traced    | This is the most significant gap. NFR-01 has no structural, behavioral, or deployment evidence in any diagram. The hardware baseline and 2-second constraint are invisible in the UML package. Should be addressed before implementation planning. |
| NFR-02 | The system architecture shall separate business logic, storage, and privacy concerns so that each layer can be unit-tested independently without instantiating the others.                                | Package diagram enforces: astranotes.service, astranotes.repository, astranotes.privacy are separate packages with downward-only dependency arrows. No upward coupling shown. | No behavioral evidence this is a structural constraint only.                                                                                            | Deployment: components shown as separate boxes (NoteService, JsonFileRepository, SecureNote+PrivacyService) confirming physical separation. | Fully Traced     | None. The package diagram is the primary evidence carrier for NFR-02 and clearly shows layer isolation.                                                                                                                                            |
| SPR-02 | The system shall catch all storage-layer exceptions (FileNotFoundException, PermissionError, JSONDecodeError) and surface clear non-technical error messages without exposing file paths or stack traces. | NoteNotFoundError and StoragePersistenceError exception classes in class diagram. No explicit error-message-format class or constraint visible.                               | Activity diagram: StoragePersistenceError shown as an outcome of the 'Write succeeds?' decision. ValidationError shown as outcome of empty title check. | No deployment evidence needed.                                                                                                              | Partially Traced | Exception classes exist and the activity diagram shows error paths. However, the constraint against exposing file paths and stack traces is not visible in any diagram it is a behavioral rule with no structural or deployment representation.    |

## 2. Traceability Metrics Summary

| Metric                                        | Count | Percentage |
|-----------------------------------------------|-------|------------|
| Total requirements reviewed                   | 8     | 100%       |
| Fully Traced                                  | 4     | 50%        |
| Partially Traced                              | 3     | 38%        |
| Weakly Traced                                 | 1     | 12%        |
| Not Traced                                    | 0     | 0%         |
| UML elements with no clear requirement reason | 2     |            |

### Reading the Metrics

50% of reviewed requirements are Fully Traced meaning structural, behavioral, and where relevant deployment evidence all exist and are consistent. 37.5% are Partially Traced the requirement is structurally visible but has a specific behavioral or constraint gap. 12.5% (NFR-01) is Weakly Traced it has no visible design evidence in any diagram type and represents the most significant pre-implementation risk in the current package.

The 2 orphan UML elements (validate\_tags, display:UI) are low risk they do not contradict any requirement but they represent design surface that outpaced the formal requirement set. Both should be either traced to a requirement or removed before Sprint 1 implementation.

## 3. Gap Analysis

The following gaps should be resolved before implementation planning begins. They are ordered by severity.

### Gap 1 NFR-01 has no design evidence (HIGH)

The 2-second performance requirement for search and listing across 500 notes is the only requirement in the reviewed set with zero structural, behavioral, or deployment evidence. There is no annotation on NoteService.search() or list\_sorted() indicating algorithmic complexity. There is no note in the deployment diagram about the hardware baseline. Before Sprint 1, one of the following should be added: (a) a performance annotation on the relevant NoteService methods in the class diagram, or (b) a constraint note in the deployment diagram referencing the hardware baseline and the 2-second target. Without this, developers have no design-level signal that performance is a first-class concern.

### Gap 2 Search flow has no activity diagram (MEDIUM)

FR-07 (search) is the second most complex flow after Create Note, yet it has no dedicated activity diagram. The private-note decrypt-on-search path where PrivacyService must be called for each private note in the result set is architecturally non-trivial and currently invisible in the behavioral diagrams. A search activity diagram should be added before Sprint 1 to clarify the flow, especially the error handling when a private note cannot be decrypted mid-search.

### Gap 3 Load failure path missing from activity diagram (LOW-MEDIUM)

FR-05 requires that corrupt or missing note files on startup are skipped with an error logged, and that remaining notes continue loading. The current activity diagram only shows the Create Note write-failure path. A startup/load activity diagram showing the corrupt-file skip logic would complete the persistence story before implementation.

#### **Gap 4 SPR-02 error-message constraint is not visible (LOW)**

The requirement that error messages must not expose file paths or stack traces is a behavioral rule with no diagram representation. The exception classes (StoragePersistenceError, NoteNotFoundError) exist in the class diagram but carry no constraint about their message format. A note or constraint annotation on these classes, or a sequence diagram showing the error surface, would close this gap.

#### **Gap 5 Two orphan UML elements without requirement backing (LOW)**

validate\_tags() in ValidationLayer and the display:UI object in the object diagram both lack a direct requirement trace. validate\_tags() should either be tied to a tag-format rule in FR-06 or removed. display:UI serves a useful explanatory purpose in the object diagram snapshot but does not correspond to a designed UI component it should be labeled as illustrative only.

## **4. How AI Helped**

---

AI was used as a drafting and cross-checking partner in three phases of building this matrix.

### **What AI helped with:**

- Generating a first-pass list of which requirements to include in the matrix. AI suggested including FR-01, FR-04, FR-05, FR-07, FR-08, NFR-01, NFR-02, and SPR-02 matching my judgment independently. The overlap gave me confidence in the selection.
- Drafting initial Class/Object Evidence entries by scanning the class diagram description I provided. AI correctly identified NoteService, NoteRepository, and PrivacyService as the primary evidence carriers for most functional requirements.
- Suggesting the gap analysis structure (ordered by severity). AI recommended ordering gaps by implementation risk rather than by requirement ID, which I agreed with and adopted.

### **What I changed or rejected:**

- AI initially marked NFR-01 as 'Partially Traced' because NoteService.list\_sorted() and search() methods exist. I downgraded it to 'Weakly Traced' method existence is not performance evidence. The requirement specifies a 2-second target and a hardware baseline; neither is represented in any diagram.
- AI marked SPR-02 as 'Fully Traced' because exception classes exist in the class diagram. I downgraded it to 'Partially Traced' the exception classes show what is raised, not the constraint on message content (no file paths, no stack traces).
- AI suggested adding FR-02 (Edit Note) and FR-03 (Delete Note) to the matrix. I excluded them because they are straightforward CRUD operations fully covered by the NoteService and NoteRepository structure, and including them would dilute focus from the higher-risk requirements.
- The status labels, gap severity ordering, and orphan element identification are my own judgment. AI did not suggest orphan elements I identified validate\_tags() and display:UI independently by asking which UML elements could not be defended from the requirement set.